SmartRetry Framework

A Selenium-Based Smart Retry Engine & AI-Driven Flaky Test Detection System

Domain: Software Testing & Quality Assurance
Core Stack: Python, Flask, Selenium WebDriver, SQLite, Google GenAI SDK
Version: 1.2.0 (Stable Release with Gemini Integration)
Target Audience: Academic Examiners, System Architects, QA Engineers

Submitted for Evaluation: Academic Project Viva-Voce Examination

1. Project Abstract & Core Foundations

1.1 Problem Statement

Modern CI/CD pipelines suffer from high regression overhead caused by *Flaky Tests*—test cases that exhibit non-deterministic behavior (passing and failing without changes in the source code). Standard test suites either fail completely when an element load is delayed or result in slow suite durations due to excessive static waits. Additionally, standard Selenium engines implicitly block on network operations, masking transition bugs and leading to false-positive pass results that hide authentic race conditions.

1.2 Project Aim

To develop an intelligent web automation framework that dynamically executes test scripts, intercepts test failures mid-transition, attempts immediate smart retries with exponential backoffs, flags non-deterministic results as 'flaky', and leverages Large Language Models (Gemini API) to deliver root-cause diagnostics and recommendations.

1.3 Key Objectives

• **Objective 1: Non-Deterministic Defect Isolation:** Bypass default page loading blocks via customized driver configurations, allowing the test suite to execute assertions during active element transition states.
• **Objective 2: Self-Healing Retry Loop:** Implement a robust retry engine containing configurable attempts, delays, and backoff multipliers, automatically updating executions schema to isolate temporary environment errors.
• **Objective 3: Low-Latency Cloud AI Integration:** Integrate the Google GenAI SDK (gemini-3.5-flash) to evaluate tracebacks, return real-time root causes, and output varying confidence scores under 2 seconds.
• **Objective 4: Live Timezone-Aware Evaluation Dashboards:** Deliver a visual user interface presenting localized execution feeds (IST), status indicators, log terminals, and evidence screenshots per attempt.
• **Objective 5: Concurrency & Transaction Management:** Prevent application conflicts and write locks during parallel test runs using SQLite Write-Ahead Logging (WAL) mode for high-throughput scaling.

2. System Architecture & Technologies

2.1 Technologies & Tools

Component	Tech/Tool Chosen	Engineering Rationale
Backend Framework	Flask 3.0.3	Lightweight, minimal routing overhead, ideal for integrating system scripts.
Automation Engine	Selenium 4.22.0	Standard API wrapper for absolute control over Chromium instances.
Database	SQLite3 (WAL Mode)	Single-file deployment with write-ahead-logging to prevent concurrency bottlenecks.
LLM Core	Google GenAI (Gemini-3.5-flash)	Natively supports new AQ-auth keys, <2s latency, high schema compliance.
Report Engine	ReportLab 4.2.2	Used to compile dynamic system execution and analytical PDF outputs.

2.2 Core System Architecture

The following diagram represents the layout flow of the architecture:

Step 1: User Request	User submits prompt or runs a test via the Flask Web Dashboard UI.
Step 2: Engine Setup	SmartRetry Engine starts Chrome with **page_load_strategy = 'none'**.
Step 3: Test Execution	Executes steps. If a step fails, it initiates the Exponential Retry Loop.
Step 4: State Analysis	Saves screenshots and attempts. Sets final status to Pass, Fail, or Flaky.
Step 5: AI Diagnostics	Google Gemini SDK reviews error trace, returning dynamic root cause & fix details.

3. Core Algorithms, Math & Implementation Details

3.1 Mathematical Formulas

3.1.1 Exponential Backoff Retry Delay

To prevent swamping the browser during resource transitions, the delay duration *d* before attempt *i* (where *i* is the retry attempt index, $i \geq 1$) is computed as:

$$d_i = D_{base} \times M^{(i - 1)}$$

Where D_{base} is the base retry delay (configured as RETRY_DELAY_SECONDS in .env, e.g., 0.5 seconds), and **M** is the growth factor (configured as RETRY_BACKOFF_MULTIPLIER, e.g., 1.0).

3.1.2 Flakiness Severity Score

The system aggregates execution logs within a sliding window of size *N* (default 10) to determine whether a test case is unstable. The flaky rating score *S* is calculated as:

$$S = [(C_{flaky} \times 0.7 + C_{failed}) / N] \times 100$$

Where C_{flaky} is the number of executions that passed on retry attempts (status = 'flaky'), and C_{failed} is the number of executions that failed all attempts. If **S = 0**, the verdict is **stable**; if **0 < S < 40**, it is **flaky**; and if **S ≥ 40**, it is flagged as **chronic**.

3.2 Step Execution Implementation

To allow genuine flaky failure detection on step verification, the Selenium webdriver page_load_strategy is set to 'none'. When an assertion step is executed, the step_executor avoids standard implicitly blocking Selenium calls and executes an instant JavaScript query directly on the DOM:

```
current_html = driver.execute_script("return document.documentElement ?
document.documentElement.innerHTML : ''") assert input_value.lower() in current_html.lower(),
f"Text '{input_value}' not found on page"
```

4. System API Documentation

The backend communicates with the UI and AI systems via Flask Blueprints. Below is the API spec:

Endpoint	Method	Description & Response Structure
`/api/generate-steps`	POST	**AI Step Generator:** Accepts user prompts, sends context to Google Gemini API (gemini-3.5-flash) and returns a JSON payload containing the Selenium steps array. **Response (JSON):** { 'steps': [{ 'action': 'open_url', 'input_value': '...', 'timeout': 10 }] }
`/api/executions`	GET	Retrieves the last 50 execution records containing status, durations, logs, and screenshots.
`/api/stats`	GET	Returns summarized stats: { 'total': 20, 'passed': 12, 'failed': 5, 'flaky': 3 }
`/ai-analysis/analyze/`	POST	Triggers a dedicated Gemini model call to analyze the traceback and execution logs, storing a dynamic confidence rating (e.g. 95%) and root cause inside the SQLite database.

5. Engineering Challenges & Solutions

1. Local LLM Latency (Ollama)

Problem: Originally, the backend called a locally running Ollama (llama3.1) instance to generate steps. However, local GPU/CPU constraints caused steps to take over 12 minutes to generate, leading to browser request timeouts.

Solution: Replaced Ollama with the cloud-based **Google Gemini API** (gemini-3.5-flash) using the official google-genai SDK (supporting both AIza and AQ auth keys). This reduced generation latency from 12 minutes to under 2 seconds.

2. False-Positive Flaky Pass Verdicts

Problem: Standard Selenium `driver.page_source` queries block execution and wait internally for the browser to finish rendering. Because of this block, tests always passed on attempt 1, failing to capture flaky network states.

Solution: Set Chrome's page_load_strategy to 'none' to prevent blocking. The framework now queries the DOM instantly using raw JavaScript document.documentElement.innerHTML, ensuring realistic transition failures.

3. Concurrency Database Locks

Problem: Simultaneous runs on the SQLite backend caused database write locks, crashing test suites.

Solution: Configured SQLite to run in **Write-Ahead Logging (WAL) Mode** and set an explicit 30-second database connection timeout to ensure thread-safety and smooth multi-user operations.

4. GitHub Push Security Scanning

Problem: Committing API keys or credentials to public codebases triggers security blockages and compromises API quotas.

Solution: Removed `.env` from tracked commits and amended git history. Added a secure `.env` mapping loaded via python-dotenv, ensuring the repository code contains no sensitive strings.

6. Simulated Examiner Viva-Voce Script

This transcript shows a simulated conversation demonstrating how to explain key details to project examiners:

Q: What is the core problem this project solves, and how does it differ from standard automation engines like Jenkins or TestNG?

A: Standard test frameworks fail a test case immediately if a network delay occurs, or they require hardcoded sleep statements that slow down the entire execution. SmartRetry solves this by implementing an active retry mechanism with exponential backoffs. If a step fails, the framework immediately retries it. If it passes on subsequent attempts, it's flagged as 'flaky' so the developer knows the application has non-deterministic behavior, rather than failing the entire build.

Q: How do you define a flaky test in your database, and what algorithm calculates this rating?

A: When a test run executes, we keep track of how many retry attempts it takes. If a test case passes but its retry count is greater than 0, we update its status in the database to 'flaky'. To calculate the overall project flakiness, we query the last 10 executions and compute a weighted severity score: Flaky runs count for 70% weight, while permanent fails count for 100%. If this ratio exceeds 40%, the test is flagged as 'chronic' indicating a severe code design flaw.

Q: I see you're using 'page_load_strategy = none'. Why is that critical for your flaky detection?

A: By default, Selenium blocks the thread and waits for the page to fire the 'load' event before executing subsequent commands. If we left it on default, we could never detect flaky conditions because Selenium would wait for the elements to fully render. By setting the page load strategy to 'none', we make Chrome return control immediately. We then run an instant JavaScript DOM query. This allows the assertion to fail on attempt 1 during the network transition, but pass on attempt 2 after a brief retry delay, which is the exact definition of a flaky test.

Q: How is AI integrated into the framework, and what models are used?

A: AI is integrated in two parts: First, a Visual Test Builder which converts natural language (e.g., 'search for shoes on Amazon') into a JSON array of Selenium steps. Second, a Failure Diagnosis engine. We call the official Google GenAI SDK to communicate with 'gemini-3.5-flash'. Gemini analyzes the stack traces and logs to output a dynamic, varying confidence rating and root cause, all formatted as raw JSON.

7. Quick-Reference Viva Questions

Q: Why choose Flask over Django for the project?

A: Flask is micro-framework oriented, making it extremely easy to plug in script run loops, background threads, and direct SQLite databases without the overhead of complex Django ORM migrations.

Q: What is the purpose of WAL mode in SQLite?

A: WAL (Write-Ahead Logging) allows concurrent reads to proceed without being blocked by ongoing write operations, which is crucial when Selenium scripts are writing logs while the user is loading the dashboard.

Q: What happens if Gemini API limit is exceeded?

A: We implemented an automated fallback mechanism: if the GenAI API returns an HTTP 429 quota error, the framework catches the exception and falls back to a regex-based heuristic analysis engine so the dashboard remains fully functional.

Q: Can this framework be used in a headless CI/CD pipeline?

A: Yes. In the `.env` configuration file, setting `HEADLESS=true` executes Chrome inside virtual memory (without rendering GUI), making it ready for integration with Jenkins, GitLab CI, or GitHub Actions.